

Exercise 08: Distributed Jacobi

Names: Tarik, Paul Dietze

This report refers to our implementation in:

- `jacobi/jacobi_mpi.cc`
- `jacobi/jacobi_omp.cc`

Only subsections (b) to (f) are included here.

(b) Decomposition and halo exchange

Where in the code:

- row decomposition: `jacobi/jacobi_mpi.cc:302-309`
- nonblocking halo exchange: `jacobi/jacobi_mpi.cc:59-67`
- blocking halo exchange for comparison: `jacobi/jacobi_mpi.cc:78-88`
- gather and verification: `jacobi/jacobi_mpi.cc:379-432`

Important snippet:

```
context->nloc = base_rows + (rank < extra_rows ? 1 : 0);
context->row_offset = 1 + rank * base_rows + std::min(rank, extra_rows);
context->up = (rank > 0) ? rank - 1 : MPI_PROC_NULL;
context->down = (rank + 1 < size) ? rank + 1 : MPI_PROC_NULL;
MPI_Isend(u + 1 * context->n, context->n, MPI_DOUBLE, context->up, 0, comm, &reqs.send_up);
MPI_Irecv(u + (context->nloc + 1) * context->n, context->n, MPI_DOUBLE, context->down, 0, comm, &reqs.recv_down);
MPI_Isend(u + context->nloc * context->n, context->n, MPI_DOUBLE, context->down, 1, comm, &reqs.send_down);
MPI_Irecv(u, context->n, MPI_DOUBLE, context->up, 1, comm, &reqs.recv_up);
For n=1024 , iterations=200 , sequential baseline 0.735 GUpdates/s :
```

ranks	blocking GUpdates/s	speedup	overlap GUpdates/s	speedup
1	0.326	0.44x	0.173	0.24x
2	0.252	0.34x	0.301	0.41x
4	0.933	1.27x	1.034	1.41x

The MPI result is correct. With 4 ranks the overlapped version is faster than the sequential reference for this problem size.

(c) Defect norm

Before computing the defect norm, we update the ghost rows again, because the first and last local rows need neighbor values. Then each rank computes its local squared defect sum. The local sums are combined with `MPI_Allreduce` , and only then we take the square root.

Where in the code:

- distributed norm: `jacobi/jacobi_mpi.cc:91-116`
- sequential norm for comparison: `jacobi/jacobi_mpi.cc:179-193`
- norm output/check: `jacobi/jacobi_mpi.cc:434-441`

Important snippet:

```
JacobiRequests reqs = halo_exchange(comm, context, u);
sync_comm(reqs);

for (int i1 = 1; i1 <= context->nloc; ++i1) {
    for (int i0 = 1; i0 < n - 1; ++i0) {
        double d = 4.0 * u[i1 * n + i0] -
            (u[i1 * n + i0 - n] + u[i1 * n + i0 - 1] +
             u[i1 * n + i0 + 1] + u[i1 * n + i0 + n]);
        local_sum += d * d;
    }
}
MPI_Allreduce(&local_sum, &global_sum, 1, MPI_DOUBLE, MPI_SUM, comm);
return sqrt(global_sum);
```

Comparison with the sequential norm:

run	MPI norm	sequential norm	difference
n=1024 , iterations=200	1.23521	1.23521	2.66e-14
n=64 , iterations=20	1.64555	1.64555	2.22e-16

The differences are only roundoff error, so the distributed norm is correct.

(d) Overlapping communication and computation

For the overlap version, the code first starts the halo exchange with nonblocking MPI calls. While the messages are in flight, it updates the inner rows. Then it waits for the halo rows and updates the boundary rows.

Where in the code:

- nonblocking exchange: `jacobi/jacobi_mpi.cc:59-67`
- wait for exchange: `jacobi/jacobi_mpi.cc:70-76`
- overlapped kernel: `jacobi/jacobi_mpi.cc:216-243`
- timing output: `jacobi/jacobi_mpi.cc:338-376`

Important snippet:

```
JacobiRequests reqs = halo_exchange(comm, context, uold);

update_rows(unew, uold, n, 2, nloc - 1);

sync_comm(reqs);

update_rows(unew, uold, n, 1, std::min(1, nloc));
update_rows(unew, uold, n, std::max(2, nloc), nloc);
For n=1024, iterations=200:
```

ranks	blocking	overlap
1	0.326	0.173
2	0.252	0.301
4	0.933	1.034

For 2 and 4 ranks, overlap is faster. For one rank it is slower because there is no real communication to hide.

(e) Bonus: MPI+X

We implemented variant `(ii)`, the hybrid version with overlap. MPI communication is done by the main thread, while OpenMP threads update the rows inside each rank.

Where in the code:

- `MPI_Init_thread`: `jacobi/jacobi_mpi.cc:270-286`
- OpenMP row update: `jacobi/jacobi_mpi.cc:166-177`
- hybrid kernel: `jacobi/jacobi_mpi.cc:245-268`

Important snippet:

```
MPI_Init_thread(&argc, &argv, MPI_THREAD_FUNNELED, &provided);
if (provided < MPI_THREAD_FUNNELED) {
    MPI_Finalize();
    return 1;
}
JacobiRequests reqs = halo_exchange(comm, context, uold);
update_rows_omp(unew, uold, n, 2, nloc - 1);
sync_comm(reqs);
update_rows_omp(unew, uold, n, 1, std::min(1, nloc));
update_rows_omp(unew, uold, n, std::max(2, nloc), nloc);
MPI_THREAD_FUNNELED is enough because only the main thread calls MPI. The OpenMP threads only compute stencil rows.
```

ranks	threads/rank	hybrid GUpdates/s
1	4	0.225
2	2	0.468
4	2	0.203

The best hybrid result here was `2 x 2`.

(f) Comparison

The previous OpenMP baseline was missing (because we didnt do the previous sheet), so we implemented a simple row-parallel OpenMP Jacobi kernel in `jacobi_omp.cc`. For the comparison, pure OpenMP uses 4 threads and pure MPI uses 4 ranks.

Where in the code:

- OpenMP baseline kernel: `jacobi/jacobi_omp.cc:65-88`
- OpenMP verification and benchmark: `jacobi/jacobi_omp.cc:139-170`
- MPI timing of blocking/overlap/hybrid: `jacobi/jacobi_mpi.cc:338-376`
- plot file: `jacobi/comparison_plot.svg`

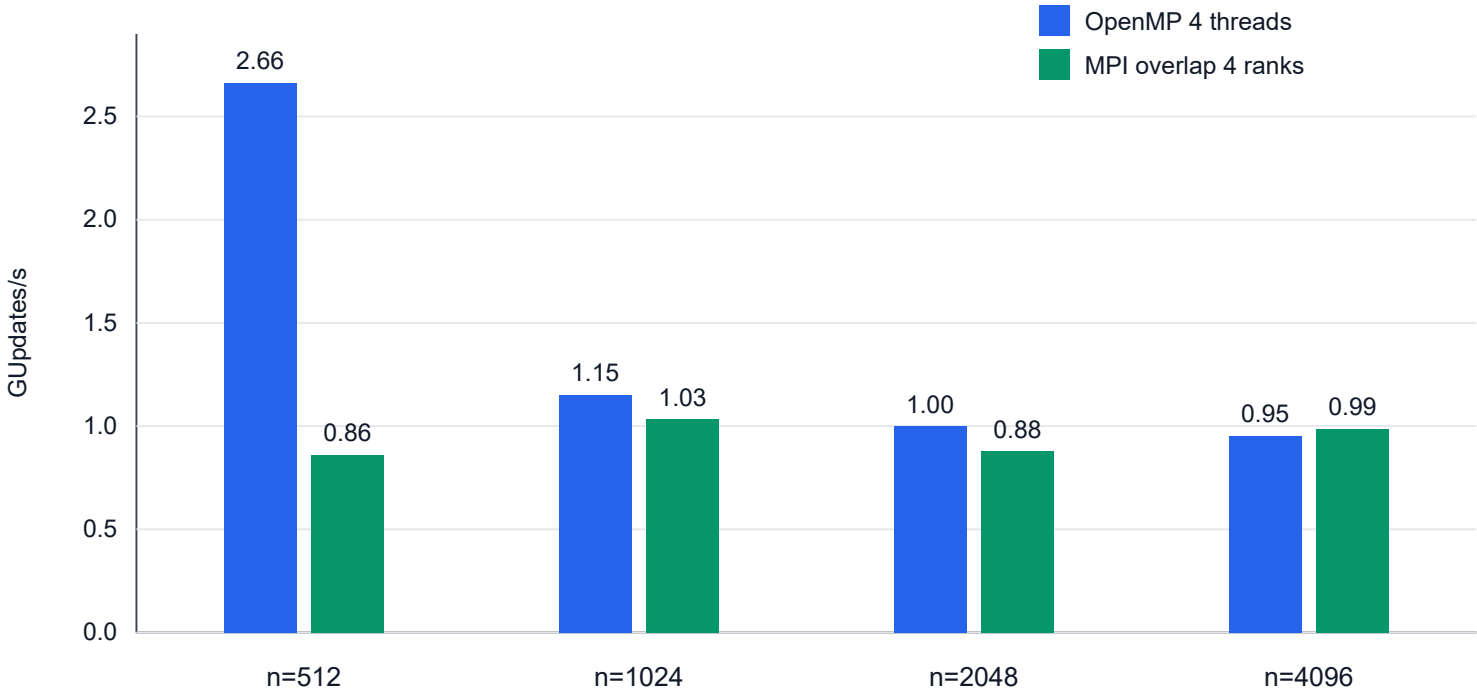
Important OpenMP snippet:

```
#pragma omp parallel
{
    for (int it = 1; it <= iterations; it++) {
#pragma omp for schedule(static)
        for (int i1 = 1; i1 < n - 1; i1++)
            for (int i0 = 1; i0 < n - 1; i0++)
                unew[i1 * n + i0] = 0.25 * (...);
    }
#pragma omp single
```

```
std::swap(uold, unew);  
}  
}
```

n	OpenMP, 4 threads	pure MPI overlap, 4 ranks
512	2.664	0.861
1024	1.152	1.034
2048	0.999	0.879
4096	0.954	0.987

OpenMP vs pure MPI overlap, 4 total workers



For small sizes OpenMP is clearly faster. This is expected because OpenMP threads can read neighbor rows directly from shared memory, while MPI has to exchange halo rows every iteration.

For larger sizes MPI gets closer, and for `n=4096` it is slightly faster in this run. The reason is the surface-to-volume effect: each MPI rank always communicates only about two boundary rows, but as the local strip gets larger it does much more computation between halo exchanges.

So the MPI overhead matters most for small strips. With larger problems, or on a real multi-node machine, MPI can become more competitive. The hybrid version from part (e) is also useful because it uses fewer MPI ranks and therefore fewer communicated strip boundaries.